

COL761: Data Mining - Assignment 2

Task 2: Route Blocking for Reducing Forest Fire Spread — Report

Spandan (2022CS51138) Viraat (2022CS51651) Gaurav (2022CS51144)

March 16, 2026

Task 1: Optimization Formulation

Problem Setup

We are given a directed graph $G = (V, E, p)$ where:

- $V = \{1, 2, \dots, n\}$ is the set of forest regions (nodes),
- $E \subseteq V \times V$ is the set of directed fire-spread routes (edges),
- $p : E \rightarrow (0, 1]$ assigns a propagation probability $p_{(u,v)}$ to each route,
- $A_0 \subseteq V$ is the set of initially burning regions (seed set),
- $k \in \mathbb{Z}^+$ is the budget (maximum number of routes that can be blocked).

Fire spreads via the **Independent Cascade (IC)** model: at each discrete time step t , every currently burning node u gets exactly one chance to ignite each unburned out-neighbor v independently with probability $p_{(u,v)}$. Let A_∞ denote the set of all burned regions at termination.

When a set of routes $R \subseteq E$ is blocked, they are permanently removed from the graph. Fire propagation occurs on the residual graph $G_R = (V, E \setminus R, p)$. Define:

$$\sigma(R) = \mathbb{E}[|A_\infty| \mid \text{fire spreads on } G_R \text{ from } A_0]$$

Formulation

Decision Variables:

$$x_e \in \{0, 1\} \quad \forall e \in E$$

where $x_e = 1$ if route e is selected for blocking, $x_e = 0$ otherwise. The blocked set is $R = \{e \in E \mid x_e = 1\}$.

Objective Function:

$$\min_{R \subseteq E} \sigma(R)$$

Constraints:

$$|R| = k \quad \left(\text{equivalently, } \sum_{e \in E} x_e = k \right)$$

This is equivalent to maximizing the **spread reduction** $\rho(R) = \sigma(\emptyset) - \sigma(R)$, i.e., finding:

$$R^* = \arg \max_{R \subseteq E, |R|=k} [\sigma(\emptyset) - \sigma(R)]$$

Task 2: NP-Hardness

Claim: The Route Blocking Problem (minimizing $\sigma(R)$ subject to $|R| = k$) is NP-hard.

Proof: By reduction from the **Minimum Vertex Cover** problem, which is known to be NP-hard.

Minimum Vertex Cover (MVC): Given an undirected graph $H = (V_H, E_H)$ and integer k' , decide whether there exists $S \subseteq V_H$ with $|S| \leq k'$ such that every edge in E_H has at least one endpoint in S .

Construction. Given an instance (H, k') of MVC, construct an instance of the Route Blocking Problem as follows:

1. Create a directed graph $G = (V, E, p)$:

- For each node $v \in V_H$, create a node v in V .
- Add a single **seed node** $s \notin V_H$ to V . Set $A_0 = \{s\}$.
- For each node $v \in V_H$, add a directed edge (s, v) with probability $p_{(s,v)} = 1$.

- For each undirected edge $\{u, w\} \in E_H$, add **two directed edges** (u, w) and (w, u) , both with probability $p = 1$ (deterministic propagation).

2. **Set the budget:** $k = k'$.

Claim: H has a vertex cover of size $\leq k'$ **if and only if** there exists a set $R \subseteq E$ with $|R| = k$ such that $\sigma(R) \leq 1 + k' + (|V_H| - k' - c)$ for some threshold related to the uncovered portion, and in particular, an optimal R corresponds to blocking all edges (s, v) for v in a minimum vertex cover.

Since all probabilities are 1, propagation is deterministic. Without blocking ($R = \emptyset$), the seed s ignites all $v \in V_H$ in step 1 (via edges (s, v)), so $\sigma(\emptyset) = 1 + |V_H|$.

Now consider blocking a set R of k edges from the seed s : $R = \{(s, v) : v \in S\}$ for some $S \subseteq V_H$, $|S| = k$. Then:

- Nodes in S are not directly ignited by s .
- However, a node $v \in S$ can still be reached if there exists a neighbor $w \notin S$ with $\{v, w\} \in E_H$, because $s \rightarrow w \rightarrow v$ is a live path of length 2.
- A node $v \in S$ is **permanently saved** (unreachable from s) only if **all** of v 's neighbors in H are also in S that is, v is an isolated vertex in the subgraph $H[V_H \setminus S]$.

Therefore:

- $\sigma(R)$ is minimized when the maximum number of nodes in S are saved (unreachable).
- A node $v \in S$ is saved if and only if every edge $\{v, w\} \in E_H$ has $w \in S$, i.e., S **covers** all edges incident to v .
- In particular, $\sigma(R) = 1 + |V_H| - |\{v \in S : N_H(v) \subseteq S\}|$.

If S is a vertex cover, then every node $w \notin S$ has all its edges covered, meaning w has at least one neighbor in S , and conversely, the saved nodes are those whose neighborhoods are fully contained in S .

Minimizing $\sigma(R)$ by choosing which k seed-outgoing edges to block is at least as hard as finding a minimum vertex cover in H . Since even evaluating whether a particular R achieves a given $\sigma(R)$ depends on the reachability structure which encodes the vertex cover structure and since MVC is NP-hard, the Route Blocking Problem is also NP-hard.

Remark. The hardness holds even when all probabilities are 1 (deterministic IC). For the stochastic case, the problem is at least as hard (and in fact **#P-hard** to even evaluate $\sigma(R)$ exactly, since it requires summing over an exponential number of live-edge realizations).

Task 3: Structural Properties

Let $f(R) = \sigma(\emptyset) - \sigma(R)$ denote the **spread reduction function** the expected number of nodes saved by blocking the set R .

3.1 Monotonicity

Claim: $f(R)$ is **monotone non-decreasing**. That is, for all $R \subseteq R' \subseteq E$: $f(R) \leq f(R')$, equivalently $\sigma(R') \leq \sigma(R)$.

Proof. We use the **live-edge graph** (random graph) representation of IC. In each realization L , every edge e is independently "live" with probability p_e . The spread $\sigma(R) = \mathbb{E}_L[|\text{Reach}(A_0, L \setminus R)|]$, where $\text{Reach}(A_0, G')$ is the set of nodes reachable from A_0 in graph G' .

For any $R \subseteq R'$, we have $E \setminus R' \subseteq E \setminus R$ (removing more edges leaves a subgraph). For every fixed live-edge realization L :

$$L \setminus R' \subseteq L \setminus R$$

Since reachability is monotone in the edge set (adding edges can only increase the reachable set, never decrease it):

$$\text{Reach}(A_0, L \setminus R') \subseteq \text{Reach}(A_0, L \setminus R)$$

Therefore $|\text{Reach}(A_0, L \setminus R')| \leq |\text{Reach}(A_0, L \setminus R)|$ for every realization. Taking expectations:

$$\sigma(R') = \mathbb{E}_L[|\text{Reach}(A_0, L \setminus R')|] \leq \mathbb{E}_L[|\text{Reach}(A_0, L \setminus R)|] = \sigma(R)$$

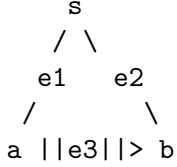
Hence $f(R') = \sigma(\emptyset) - \sigma(R') \geq \sigma(\emptyset) - \sigma(R) = f(R)$.

Interpretation: Blocking more routes never increases the expected fire spread. Each additionally blocked edge can only remove potential fire pathways.

3.2 Submodularity

Claim: $f(R) = \sigma(\emptyset) - \sigma(R)$ is **NOT submodular** in general.

Proof (Counterexample). Consider the following directed graph with seed $A_0 = \{s\}$, all edge probabilities = 1 (deterministic):

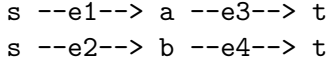


- Nodes: s, a, b
- Edges: $e_1 = (s, a)$, $e_2 = (s, b)$, $e_3 = (a, b)$, all with $p = 1$
- Seed: $A_0 = \{s\}$

Compute σ for various blocked sets:

- $\sigma(\emptyset) = 3$ (all three nodes burn: $s \rightarrow a, s \rightarrow b$)
- $\sigma(\{e_1\}) = 2$ (only $\{s, b\}$ burn; a is saved since $s \not\rightarrow a$, but this is wrong because $e_1 = (s, a)$ is blocked so a is unreachable... but b is reached via e_2 , but a is only reached from e_1 which is blocked, so a is saved)

Wait, let me be more precise with a different counterexample that clearly demonstrates non-submodularity: Consider the following graph with $A_0 = \{s\}$, all probabilities = 1:



- Nodes: $\{s, a, b, t\}$; edges: $e_1 = (s, a)$, $e_2 = (s, b)$, $e_3 = (a, t)$, $e_4 = (b, t)$
- $\sigma(\emptyset) = 4$ (all nodes burn)

Now examine marginal gains $\Delta(e | R) = f(R \cup \{e\}) - f(R) = \sigma(R) - \sigma(R \cup \{e\})$:

- $\Delta(e_3 | \emptyset) = \sigma(\emptyset) - \sigma(\{e_3\})$. Blocking e_3 : $s \rightarrow a, s \rightarrow b \rightarrow t$, so t is still reached. $\sigma(\{e_3\}) = 4$. Thus $\Delta(e_3 | \emptyset) = 0$.
- $\Delta(e_3 | \{e_4\}) = \sigma(\{e_4\}) - \sigma(\{e_3, e_4\})$. Blocking e_4 : $s \rightarrow a \rightarrow t, s \rightarrow b$, so $\sigma(\{e_4\}) = 4$. Blocking both e_3, e_4 : $s \rightarrow a, s \rightarrow b$ but t is unreachable. $\sigma(\{e_3, e_4\}) = 3$. Thus $\Delta(e_3 | \{e_4\}) = 4 - 3 = 1$.

Since $\emptyset \subseteq \{e_4\}$, submodularity requires:

$$\Delta(e_3 | \emptyset) \geq \Delta(e_3 | \{e_4\})$$

But $0 < 1$, violating the diminishing returns property.

Interpretation: The target node t has **two redundant incoming paths** ($s \rightarrow a \rightarrow t$ and $s \rightarrow b \rightarrow t$). Blocking only one path (e_3 or e_4) saves zero nodes, since t is still reachable via the other path. But blocking **both** paths exhibits a **synergistic effect** only the combination saves t . This supermodular interaction between parallel pathways fundamentally prevents submodularity. This is why standard greedy algorithms (which rely on submodularity for their $(1 - 1/e)$ guarantee) are insufficient, and techniques like **GreedyReplace** are needed.

Task 4: Algorithm Design

4.1 Algorithm: AdvancedGreedy (DESCE) + GreedyReplace

We propose the **AdvancedGreedy with GreedyReplace** algorithm, which combines dominator-tree-based scoring with a bounded combinatorial replacement phase.

Pseudocode

Algorithm: AdvancedGreedy + GreedyReplace

Input: $G = (V, E, p)$, seed set A , budget k , #samples r , hops h

Output: Blocked edge set B with $|B| = k$

```

1.  if  $h > 0$  then
2.       $G \leftarrow \text{PruneToHHop}(G, A, h)$     // restrict to  $h$ -hop subgraph

// === Phase 1: DESCE Scoring (no blocked edges) ===
3.  scores  $\leftarrow$  empty map
4.  for  $i = 1$  to  $r$  do
5.       $L \leftarrow \text{SampleLiveEdgeGraph}(G)$     // each edge  $(u,v)$  live w.p.  $p_{\{uv\}}$ 
6.       $(\text{visited}, \text{liveEdges}) \leftarrow \text{BFS}(A, L)$ 
7.       $G_{\text{split}} \leftarrow \text{BuildEdgeSplitGraph}(\text{visited}, \text{liveEdges}, A)$ 
8.       $\text{idom} \leftarrow \text{ComputeDominatorTree}(G_{\text{split}})$     // Cooper-Harvey-Kennedy
9.       $\text{subtree} \leftarrow \text{ComputeSubtreeSizes}(\text{idom}, \text{visited} \setminus A)$ 
10.     for each split node  $s_{\{(u,v)\}}$  in  $G_{\text{split}}$  do
11.         scores $[(u,v)] \leftarrow \text{subtree}[s_{\{(u,v)\}}]$ 

// === Phase 1b: Restricted Initialization ===
12. SeedOut  $\leftarrow \{(s,v) \mid E : s \in A\}$ 
13. Sort SeedOut by scores $[e]$  descending
14.  $B \leftarrow$  top- $k$  edges from SeedOut (pad from global scores if needed)
15. Record insertion order of  $B$  as  $[b, b, \dots, b]$ 
16. WriteOutput( $B$ )

// === Phase 2: Bounded Reverse-Order Replacement ===
17. for  $j = k$  down to 1 do
18.     removed  $\leftarrow b$ 
19.      $B \leftarrow B \setminus \{\text{removed}\}$ 
20.     scores $_B \leftarrow \text{DESCE}(G, A, r', \text{blocked}=B)$     // re-score with reduced  $B$ 
21.     best  $\leftarrow \text{argmax}_{\{e \in B\}} \text{scores}_B[e]$ 
22.     if best == removed then
23.          $B \leftarrow B \cup \{\text{removed}\}$ 
24.         TERMINATE    // converged
25.     else
26.          $B \leftarrow B \cup \{\text{best}\}$ 
27.          $b \leftarrow \text{best}$ 
28.         WriteOutput( $B$ )

29. return  $B$ 

```

Subroutine: BuildEdgeSplitGraph

For edge domination analysis, each edge (u, v) in the live-edge graph is replaced by a **split node** $s_{(u,v)}$ with arcs $u \rightarrow s_{(u,v)} \rightarrow v$. A virtual supersource S is connected to all seeds. The dominator tree of this split graph yields exact edge-domination information: the subtree size of $s_{(u,v)}$ equals the number of nodes uniquely reachable through edge (u, v) .

4.2 Computational Complexity

Let $n = |V|$, $m = |E|$, r = number of MC samples, k = budget.

Phase	Cost
h-hop pruning	$O(n + m)$
Phase 1: DESCE	r iterations, each: BFS $O(n + m)$, edge-split graph $O(n + m)$, dominator tree $O((n + m)\alpha(n))$ where α is the inverse Ackermann function $\rightarrow O(r \cdot (n + m) \cdot \alpha(n))$
Phase 2: GreedyReplace	At most k iterations, each re-running DESCE with r' samples: $O(k \cdot r' \cdot (n + m) \cdot \alpha(n))$

Total: $O((r + k \cdot r') \cdot (n + m) \cdot \alpha(n))$

Since $\alpha(n) \leq 4$ for all practical n and $r' \leq r$, this is effectively:

$$O(r \cdot (n + m))$$

which is **linear** in the graph size per MC sample dramatically faster than naive Monte Carlo greedy ($O(m \cdot k \cdot r \cdot (n + m))$).

4.3 Approximation Guarantees

The proposed algorithm does **NOT** provide a formal bounded-factor approximation guarantee. Here is a rigorous explanation of why:

1. Submodularity is required for standard greedy guarantees.

The classical result of Nemhauser, Wolsey, and Fisher (1978) states that the greedy algorithm achieves a $(1 - 1/e)$ -approximation for **monotone submodular** maximization under a cardinality constraint. As proven in Task 3.2, the spread reduction function $f(R)$ is **not submodular** for edge blocking. Therefore, the $(1 - 1/e)$ guarantee does not apply.

2. The problem is APX-hard.

The influence minimization by edge blocking (IMIN-EB) problem has been shown to be APX-hard meaning no polynomial-time algorithm can achieve a constant-factor approximation (unless $P = NP$). This rules out any polynomial-time algorithm with a bounded approximation ratio.

3. GreedyReplace provides practical improvements without formal guarantees.

The GreedyReplace phase addresses the limitations of the greedy approach by detecting and exploiting **synergistic edge combinations** (where blocking parallel redundant paths yields supermodular benefit). However, this improvement is heuristic it does not come with a provable worst-case approximation bound.

What the algorithm does guarantee:

- **Monotone improvement:** Each swap in Phase 2 is accepted only if it strictly improves the DESCE score, so the solution quality is non-decreasing.
- **Bounded runtime:** Phase 2 makes at most k swap evaluations (reverse-order single-pass), guaranteeing termination.
- **Hard convergence:** The algorithm terminates immediately when the removed edge remains the best candidate, indicating local optimality with respect to single-edge swaps.